Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19-03-2026**	**Enrolment No:92301733024**

Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:
Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:
Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:
Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly
actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word	stemmed words
0	connect	connect
'I	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19-03-2026**	**Enrolment No:92301733024**

Stopword Removal

Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", ‘‘the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?", you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Program (Code):

To be attached with

1. Tokenization
2. Filteration
3. StopWords Removal
4. PoS Tagging
5. Noun Phrase Chunking
6. Dependancy Parsing

```
!pip install nltk spacy
!python -m spacy download en_core_web_sm
import nltk
import spacy
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk import pos_tag, ne_chunk

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')
text = """Natural language processing is a part of artificial intelligence.
It helps computers understand human language.
Students are learning NLP in their courses.
Text preprocessing is the first step in NLP.
Tokenization breaks text into words.
```

```
Stopwords removal improves efficiency.
Stemming reduces words to their base form.
POS tagging identifies parts of speech.
Chunking groups words into phrases.
Dependency parsing shows relationships between words."""
import nltk
nltk.download('punkt_tab')
tokens = word_tokenize(text)
print("Tokenization:\n", tokens)
```

```
Tokenization:
 ['Natural', 'language', 'processing', 'is', 'a', 'part', 'of', 'artificial', 'intelligence', '.', 'It', 'helps', 'computers', 'understand', 'human', 'language', '.', 'Students', '
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
stop_words = set(stopwords.words('english'))
filtered_tokens = [w for w in tokens if w.lower() not in stop_words]
print("\nAfter Stopword Removal:\n", filtered_tokens)
```

```
After Stopword Removal:
 ['Natural', 'language', 'processing', 'part', 'artificial', 'intelligence', '.', 'helps', 'computers', 'understand', 'human', 'language', '.', 'Students', 'learning', 'NLP', 'cou
```

```
ps = PorterStemmer()
stemmed_words = [ps.stem(w) for w in filtered_tokens]
print("\nStemmed Words:\n", stemmed_words)
```

```
Stemmed Words:
 ['natur', 'languag', 'process', 'part', 'artifici', 'intellig', '.', 'help', 'comput', 'understand', 'human', 'languag', '.', 'student', 'learn', 'nlp', 'cours', '.', 'text', 'pre
```

```
import nltk
nltk.download('averaged_perceptron_tagger_eng')
pos_tags = pos_tag(filtered_tokens)
print("\nPOS Tags:\n", pos_tags)
```

```
POS Tags:
 [('Natural', 'JJ'), ('language', 'NN'), ('processing', 'VBG'), ('part', 'NN'), ('artificial', 'JJ'), ('intelligence', 'NN'), ('.', '.'), ('helps', 'VBZ'), ('computers', 'NNS'), ('
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

```
grammar = "NP: {<DT>?<JJ>*<NN>}"
chunk_parser = nltk.RegexpParser(grammar)
chunk_tree = chunk_parser.parse(pos_tags)
print("\nChunking (Noun Phrases):")
print(chunk_tree)
```

```
Chunking (Noun Phrases):
(S
  (NP Natural/JJ language/NN)
  processing/VBG
  (NP part/NN)
  (NP artificial/JJ intelligence/NN)
  ./.
  helps/VBZ
  computers/NNS
  understand/VBP
  (NP human/JJ language/NN)
  ./.
  Students/NNS
  learning/VBG
  NLP/NNP
  courses/NNS
  ./.
  Text/NNP
  preprocessing/VBG
  (NP first/JJ step/NN)
  NLP/NNP
  ./.
  Tokenization/NNP
  breaks/IN
  text/JJ
  words/NNS
  ./.
  Stopwords/NNS
  removal/JJ
  improves/NNS
  (NP efficiency/NN)
  ./.
  Stemming/VBG
```

```
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)
print("\nDependency Parsing:")
for token in doc:
    print(token.text, "->", token.dep_, "->", token.head.text)
```

```
Dependency Parsing:
Natural -> amod -> language
language -> compound -> processing
processing -> nsubj -> is
is -> ROOT -> is
a -> det -> part
part -> attr -> is
of -> prep -> part
artificial -> amod -> intelligence
intelligence -> pobj -> of
. -> punct -> is

 -> dep -> .
It -> nsubj -> helps
helps -> ROOT -> helps
computers -> nsubj -> understand
understand -> ccomp -> helps
human -> amod -> language
language -> dobj -> understand
. -> punct -> helps

 -> dep -> .
Students -> nsubj -> learning
are -> aux -> learning
learning -> ROOT -> learning
NLP -> dobj -> learning
in -> prep -> learning
their -> poss -> courses
courses -> pobj -> in
. -> punct -> learning
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19-03-2026**	**Enrolment No:92301733024**

Observation:

Text preprocessing is the first step in NLP. Tokenization splits text into words. Stopword removal removes unnecessary words. Stemming reduces words to their root form. POS tagging identifies parts of speech. Chunking groups words into phrases. Dependency parsing shows relationship between words.

Post Lab Exercise:

1. Take your own document of 10 sentences and perform the same tasks. Attach code and screenshot of your output.

text = """I am learning Natural Language Processing.
It helps in understanding human language.
I like working on machine learning projects.
Text preprocessing is very important in NLP.
Tokenization splits sentences into words.
Stopword removal removes common words.
Stemming reduces words to base form.
POS tagging identifies grammar of words.
Chunking forms meaningful phrases.
Dependency parsing shows word relations."""

```
import nltk
nltk.download('punkt_tab')
tokens = word_tokenize(text)
print("Tokenization:\n", tokens)

[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Tokenization:
 ['I', 'am', 'learning', 'Natural', 'Language', 'Processing', '.', 'It', 'helps', 'in', 'understanding', 'human', 'language', '.', 'I', 'like', 'working', 'on', 'machine', 'learning', 'p
```

```
stop_words = set(stopwords.words('english'))
filtered_tokens = [w for w in tokens if w.lower() not in stop_words]
print("\nAfter Stopword Removal:\n", filtered_tokens)

After Stopword Removal:
 ['learning', 'Natural', 'Language', 'Processing', '.', 'helps', 'understanding', 'human', 'language', '.', 'like', 'working', 'machine', 'learning', 'projects', '.', 'Text', 'preprocess
```

```
ps = PorterStemmer()
stemmed_words = [ps.stem(w) for w in filtered_tokens]
print("\nStemmed Words:\n", stemmed_words)

Stemmed Words:
 ['learn', 'natur', 'languag', 'process', '.', 'help', 'understand', 'human', 'languag', '.', 'like', 'work', 'machin', 'learn', 'project', '.', 'text', 'preprocess', 'import', 'nlp', '.
```

```
import nltk
nltk.download('averaged_perceptron_tagger_eng')
pos_tags = pos_tag(filtered_tokens)
print("\nPOS Tags:\n", pos_tags)

[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.

POS Tags:
 [('learning', 'VBG'), ('Natural', 'NNP'), ('Language', 'NNP'), ('Processing', 'NNP'), ('.', '.'), ('helps', 'VBZ'), ('understanding', 'VBG'), ('human', 'JJ'), ('language', 'NN'), ('.',
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19-03-2026**	**Enrolment No:92301733024**

```python
grammar = "NP: {<DT>?<JJ>*<NN>}"
chunk_parser = nltk.RegexpParser(grammar)
chunk_tree = chunk_parser.parse(pos_tags)
print("\nChunking (Noun Phrases):")
print(chunk_tree)
```

```
Chunking (Noun Phrases):
(S
  learning/VBG
  Natural/NNP
  Language/NNP
  Processing/NNP
  ./.
  helps/VBZ
  understanding/VBG
  (NP human/JJ language/NN)
  ./.
  like/IN
  working/VBG
  (NP machine/NN)
  learning/VBG
  projects/NNS
  ./.
  Text/NNP
  preprocessing/VBG
  important/JJ
  NLP/NNP
  ./.
  Tokenization/NNP
  splits/VBZ
  sentences/NNS
```

```python
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

print("\nDependency Parsing:")
for token in doc:
    print(token.text, "->", token.dep_, "->", token.head.text)
```

```
 -> dep -> .
Text -> compound -> preprocessing
preprocessing -> nsubj -> is
is -> ROOT -> is
very -> advmod -> important
important -> acomp -> is
in -> prep -> is
NLP -> pobj -> in
. -> punct -> is

 -> dep -> .
Tokenization -> nsubj -> splits
splits -> ROOT -> splits
sentences -> dobj -> splits
into -> prep -> splits
words -> pobj -> into
. -> punct -> splits

 -> dep -> .
Stopword -> compound -> removal
removal -> nsubj -> removes
removes -> ROOT -> removes
common -> amod -> words
words -> dobj -> removes
. -> punct -> removes
```

2. Write your observation for stemming and lemmatization you obtained for the sentences.
   Stemming removes suffixes and gives root words. Sometimes it produces incorrect words (like “troubl”). Lemmatization gives proper meaningful base words. Lemmatization is more accurate than stemming.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19-03-2026**	**Enrolment No:92301733024**